МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Инженерно-экономический факультет
Кафедра экономической информатики

Отчет
по лабораторной работе №2

на тему «JDBC»
по курсу «Современные технологии серверной разработки»

Выполнил: Р. В. Тамашевский
Студент группы 472303

Проверила: Е. А. Мельникова

Минск 2026

Цель: приобрести навыки работы с основными способами взаимодействия с базами данных на языке Java.

Задание: В пакете statement реализуем семь заданий для предметной области «Маркетплейс услуг». Задание: сделать выборку по всем таблицам базы данных; сделать выборку исполнителей, отсортировав их по имени и фамилии; сделать выборку услуг с ценой выше заданной, отсортировав по стоимости; обновить статус случайного заказа конкретного клиента; сделать выборку всех услуг, категория которых начинается на определённую букву; обновить телефоны клиентов, чьи заказы находятся в определённом статусе, с помощью подзапроса; сделать выборку услуг, сгруппировав её по категориям, и вывести количество услуг для каждой категории с помощью подзапроса. Логику всех выборок разместим в методе getAllInfo() класса StatementAction.

В пакете prepared_statement добавим две новые сущности – Car и RepairJob (по 5 полей каждая) для предметной области «Автосервис». Задание: произвести над ними четыре базовые операции – добавление новых записей (INSERT), поиск ремонтов по идентификатору механика (SELECT с условием WHERE), обновление стоимости ремонта (UPDATE) и удаление старых записей о ремонтах (DELETE).

Ход выполнения работы:

Первая предметная область – Маркетплейс услуг – моделирует цифровую платформу, на которой исполнители предлагают свои услуги клиентам. Основными сущностями являются исполнители, хранящие информацию о специалисте, его специализации и рейтинге; услуги, описывающие конкретные виды работ с указанием категории и стоимости; клиенты, представляющие пользователей платформы с их контактными данными; и заказы, фиксирующие факт обращения клиента за конкретной услугой с указанием даты и статуса выполнения. Связи между сущностями реализованы через внешние ключи: заказ ссылается на клиента и услугу, а услуга, в свою очередь, связана с исполнителем.

Вторая предметная область – Автосервис – описывает процессы технического обслуживания и ремонта транспортных средств в сервисном центре. Ключевыми сущностями выступают механики, содержащие данные о сотруднике, его стаже и специализации; автомобили, хранящие информацию о марке, модели, годе выпуска и государственном номере транспортного средства; и ремонты, представляющие журнал выполненных работ, который связывает конкретный автомобиль с механиком и фиксирует дату проведения ремонта и его стоимость.

Создаем базу данных:



Напишем и выполним запрос написания таблиц:

```sql
CREATE TABLE executors (
    id BIGSERIAL PRIMARY KEY,
    first_name TEXT NOT NULL,
    last_name TEXT,
    specialization TEXT,
    rating INT
);

CREATE TABLE services (
    id BIGSERIAL PRIMARY KEY,
    title TEXT NOT NULL,
    executor_id BIGINT NOT NULL,
    price INT NOT NULL,
    category TEXT,
    FOREIGN KEY (executor_id) REFERENCES executors(id)
);

CREATE TABLE clients (
    id BIGSERIAL PRIMARY KEY,
    first_name TEXT NOT NULL,
    phone TEXT,
    email TEXT
);

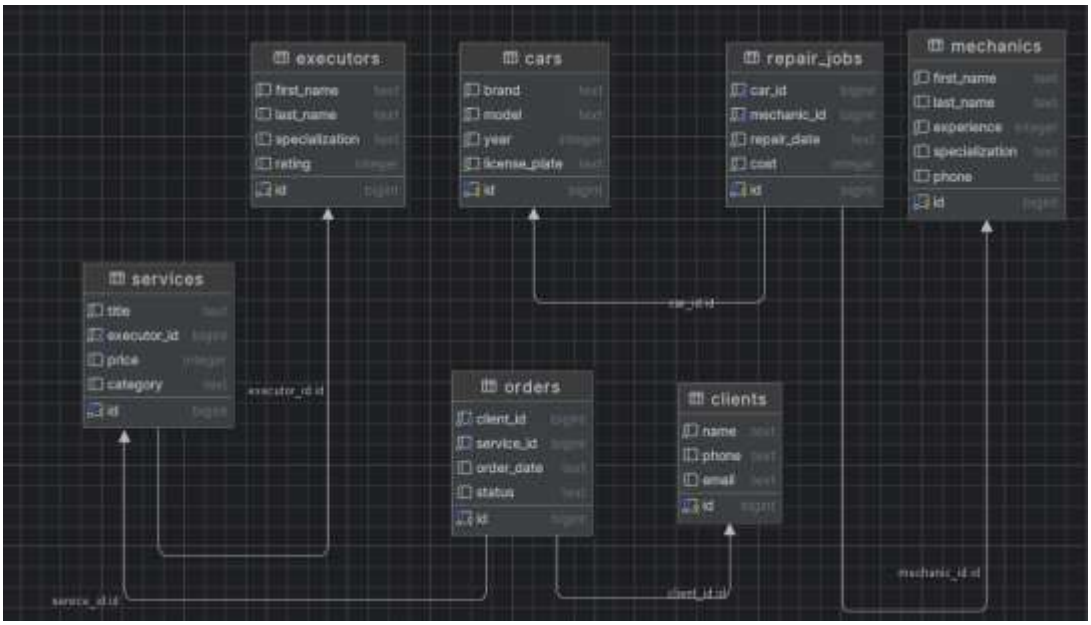
CREATE TABLE orders (
    id BIGSERIAL PRIMARY KEY,
    client_id BIGINT NOT NULL,
    service_id BIGINT NOT NULL,
    order_date TEXT NOT NULL,
    status TEXT,
    FOREIGN KEY (client_id) REFERENCES clients(id),
    FOREIGN KEY (service_id) REFERENCES services(id)
);

CREATE TABLE mechanics (
    id BIGSERIAL PRIMARY KEY,
    first_name TEXT NOT NULL,
    last_name TEXT,
    experience INT NOT NULL,
    specialization TEXT,
    phone TEXT
);

CREATE TABLE cars (
    id BIGSERIAL PRIMARY KEY,
    brand TEXT NOT NULL,
    model TEXT NOT NULL,
    year INT NOT NULL,
    license_plate TEXT NOT NULL
);

CREATE TABLE repair_jobs (
    id BIGSERIAL PRIMARY KEY,
    car_id BIGINT NOT NULL,
    mechanic_id BIGINT NOT NULL,
    repair_date TEXT NOT NULL,
    cost INT NOT NULL,
    FOREIGN KEY (car_id) REFERENCES cars(id),
    FOREIGN KEY (mechanic_id) REFERENCES mechanics(id)
);
```

В результате выполнения представленного SQL-скрипта в СУБД PostgreSQL была создана база данных jdbc_lab, содержащая 7 таблиц, объединённых в две логические группы по предметным областям.

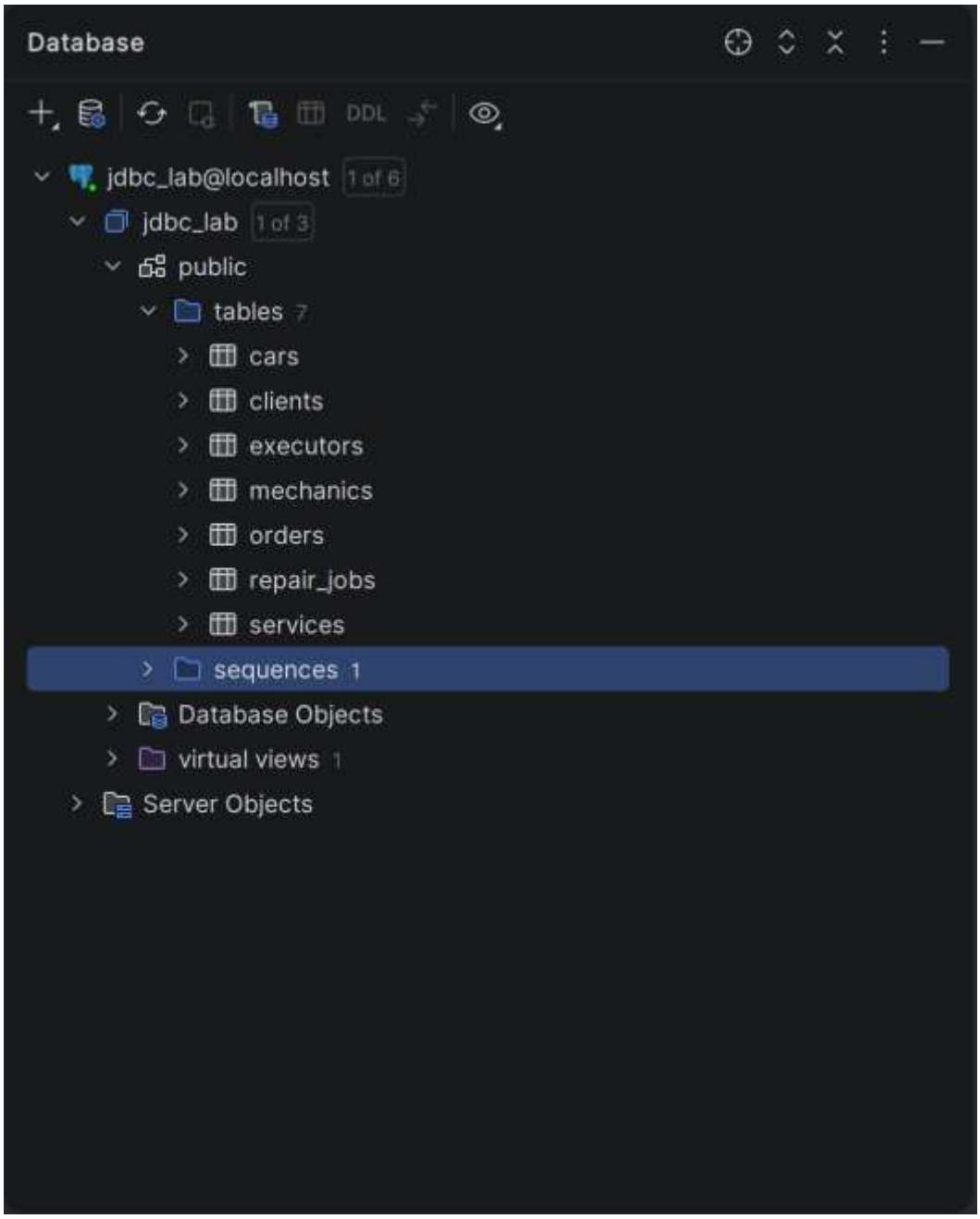


Создаём проект на языке Java в среде разработки IntelliJ IDEA с использованием системы сборки Maven. Maven позволяет автоматически управлять зависимостями (библиотеками) и структурой проекта.

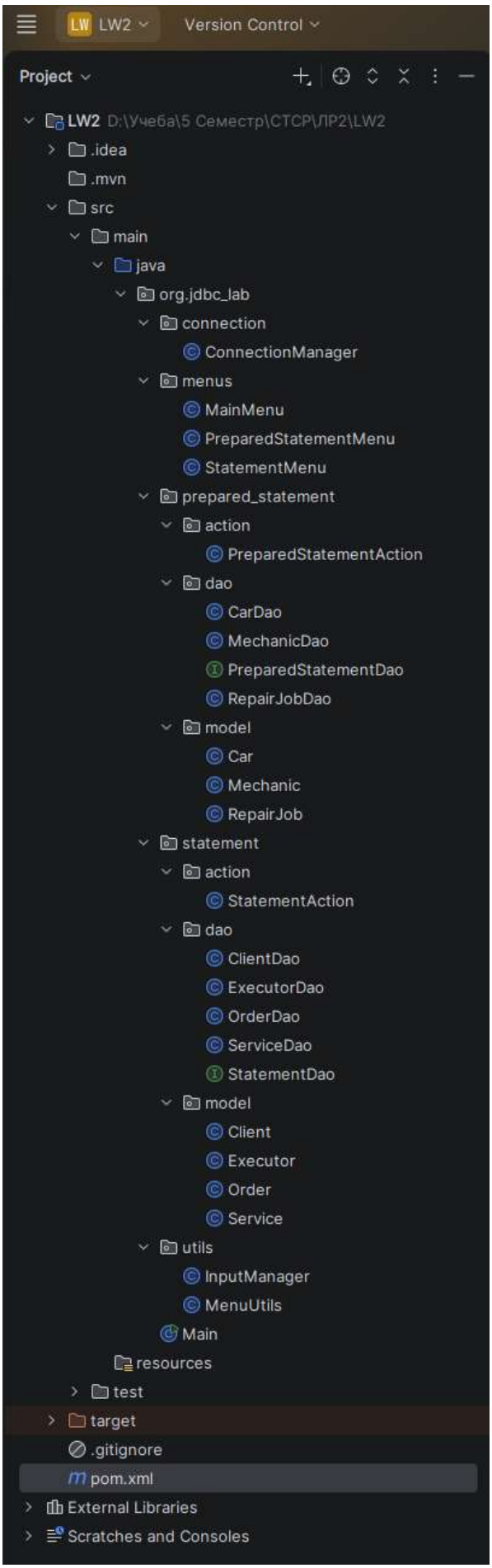
Настроим файл pom.xml. В секцию <dependencies> были добавим две необходимые библиотеки: драйвер JDBC для PostgreSQL, который обеспечивает взаимодействие Java-приложения с СУБД PostgreSQL, библиотека Lombok для автоматической генерации шаблонного кода.

```
<dependencies>
    <!-- Source: https://mvnrepository.com/artifact/org.postgresql/postgresql -->
    <dependency>
        <groupId>org.postgresql</groupId>
        <artifactId>postgresql</artifactId>
        <version>42.7.11</version>
        <scope>compile</scope>
    </dependency>
    <!-- Source: https://mvnrepository.com/artifact/org.projectlombok/lombok -->
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <version>1.18.46</version>
        <scope>compile</scope>
    </dependency>
</dependencies>
```

Подключимся к базе данных непосредственно через встроенный инструмент Database. В правой части экрана откроем одноимённую панель, где с помощью кнопки добавления создадим новый источник данных



После подключения к базе данных создадим структуру пакетов проекта в директории src/main/java. Корневым пакетом объявим org.jdbc_lab, внутри которого сформируем подпакеты connection, model, dao, action, menus и utils. Такая структура соответствует паттерну DAO и разделяет ответственность между слоями приложения.


LW2
Version Control
Project
LW2 D:\Учеба\5 Семестр\СТСР\ЛР2\LW2
.idea
.mvn
src
main
java
org.jdbc_lab
connection
ConnectionManager
menus
MainMenu
PreparedStatementMenu
StatementMenu
prepared_statement
action
PreparedStatementAction
dao
CarDao
MechanicDao
PreparedStatementDao
RepairJobDao
model
Car
Mechanic
RepairJob
statement
action
StatementAction
dao
ClientDao
ExecutorDao
OrderDao
ServiceDao
StatementDao
model
Client
Executor
Order
Service
utils
InputManager
MenuUtils
Main
resources
test
target
.gitignore
pom.xml
External Libraries
Scratches and Consoles

Далее в пакете connection реализуем класс ConnectionManager по паттерну Singleton. Он будет хранить параметры подключения (URL, логин, пароль) и предоставлять статические методы getConnection() для получения активного соединения и closeConnection() для его закрытия. Все операции с базой данных в проекте проходят через этот класс.

```java
package org.jdbc_lab.connection;

import ...

public class ConnectionManager {  45 usages
    private static final Logger logger = Logger.getLogger(ConnectionManager.class.getName());  6 usages
    private static Connection connection = null;  6 usages

    public static Connection getConnection() {  37 usages
        if (connection == null) {
            try {
                Class.forName( className: "org.postgresql.Driver");
            } catch (ClassNotFoundException e) {
                closeConnection();
                logger.severe( msg: "Драйвер для БД не найден: " + e.getMessage());
            }

            final String user = "postgres";
            final String password = "1";
            final String url = "jdbc:postgresql://localhost:5432/jdbc_lab";

            try {
                connection = DriverManager.getConnection(url, user, password);
                logger.info( msg: "Соединение с БД установлено успешно!");
            } catch (SQLException e) {
                closeConnection();
                logger.severe( msg: "Соединение с БД установить не получилось: " + e.getMessage());
            }

        }
        return connection;
    }

    public static void closeConnection(){  2 usages
        if (connection != null){
            try {
                connection.close();
                logger.info( msg: "Соединение с БД закрыто успешно!");
            }
            catch (SQLException e){
                logger.severe( msg: "Соединение с БД закрыть не получилось: " + e.getMessage());
            }
            finally {
                connection = null;
            }
        }
        else {
            logger.info( msg: "Обнаружен запрос закрытия уже закрытого соединения с БД!");
        }
    }

}
```

Точкой входа в приложение создадим класс Main. В его методе main() сначала вызовем ConnectionManager.getConnection() для установки соединения с БД, затем запустим главное меню, а в конце работы программы обязательно закроем соединение.

```java
package org.jdbc_lab;

import org.jdbc_lab.menus.MainMenu;

public class Main {
    public static void main(String[] args) {
        MainMenu mainMenu = new MainMenu();
        mainMenu.print();
    }
}
```

Чтобы обеспечить удобный и безопасный ввод данных от пользователя, создадим в пакете utils класс InputManager. В нем реализуем методы для считывания целых чисел, дробных чисел и строк с обработкой возможных ошибок ввода. Также добавим класс MenuUtils для автоматического формирования заголовков меню в консоли.

```java
package org.jdbc_lab.utils;

import java.util.List;

public class MenuUtils {  no usages
    public static String getHeader(final String title, final List<String> options){
        StringBuilder header = new StringBuilder("\n\n[" + title + "]\n\n");

        for (int i = 0; i < options.size(); i++){
            header.append(i+1).append(") ").append(options.get(i)).append(".\n");
        }

        header.append("0) Выход.\n").append("\nВыберите: ");

        return header.toString();
    }
}
```

```java
package org.jdbc_lab.utils;

import ...

public class InputManager {  6 usages
    public static int getNextInt(){  1 usage
        Scanner scanner = new Scanner(System.in);
        while (true){
            try {
                return scanner.nextInt();
            }
            catch (InputMismatchException e){
                scanner.nextLine();
                System.out.print("Необходимо целое. Попробуй снова: ");
            }
        }
    }

    public static int getNextIntInRange(int min, int max){  1 usage
        Scanner scanner = new Scanner(System.in);
        while (true){
            int input = getNextInt();

            if (input < min || input > max){
                System.out.print("Введённое находится за пределами. Попробуй снова: ");
            }
            else {
                return input;
            }
        }
    }

    public static long getNextLong(){  no usages
        Scanner scanner = new Scanner(System.in);

        while (true){
            try {
                return scanner.nextLong();
            }
            catch (InputMismatchException e){
                scanner.nextLine();
                System.out.print("Необходимо длинное целое. Попробуй снова: ");

            }
        }
    }
```

Для организации навигации в пакете menus напишем класс MainMenu. Он будет выводить в консоль список доступных режимов работы (Statement и PreparedStatement) и передавать управление соответствующим подменю.



```
package org.jdbc_lab.menus;

import org.jdbc_lab.menus.PreparedStatementMenu;
import org.jdbc_lab.menus.StatementMenu;

import java.util.Scanner;

public class MainMenu {
    private final StatementMenu statementMenu = new StatementMenu();
    private final PreparedStatementMenu preparedStatementMenu = new PreparedStatementMenu();
    private final Scanner scanner = new Scanner(System.in);

    public void print() {
        while (true) {
            System.out.println("\n=== ГЛАВНОЕ МЕНЮ ===");
            System.out.println("1. Маркетплейс услуг (Statement)");
            System.out.println("2. Автосервис (PreparedStatement)");
            System.out.println("3. Выйти из программы");
            System.out.print("Выберите пункт: ");

            try {
                int choice = scanner.nextInt();
                scanner.nextLine(); // очищаем буфер

                switch (choice) {
                    case 1:
                        statementMenu.print();
                        break;
                    case 2:
                        preparedStatementMenu.print();
                        break;
                    case 3:
                        System.out.println("До свидания!");
                        return;
                    default:
                        System.out.println("Неверный пункт меню! Попробуйте снова.");
                }
            } catch (Exception e) {
                System.out.println("Ошибка ввода! Пожалуйста, введите число(1, 2 или 3.");
                scanner.nextLine();
            }
        }
    }
}
```

В пакете model создадим POJO-классы, представляющие таблицы базы данных в виде Java-объектов. Для сущностей Author, Book, Shop, ShopBook и Student добавим аннотацию @Data от Lombok, которая автоматически генерирует геттеры, сеттеры и метод toString(), что сокращает объём шаблонного кода.



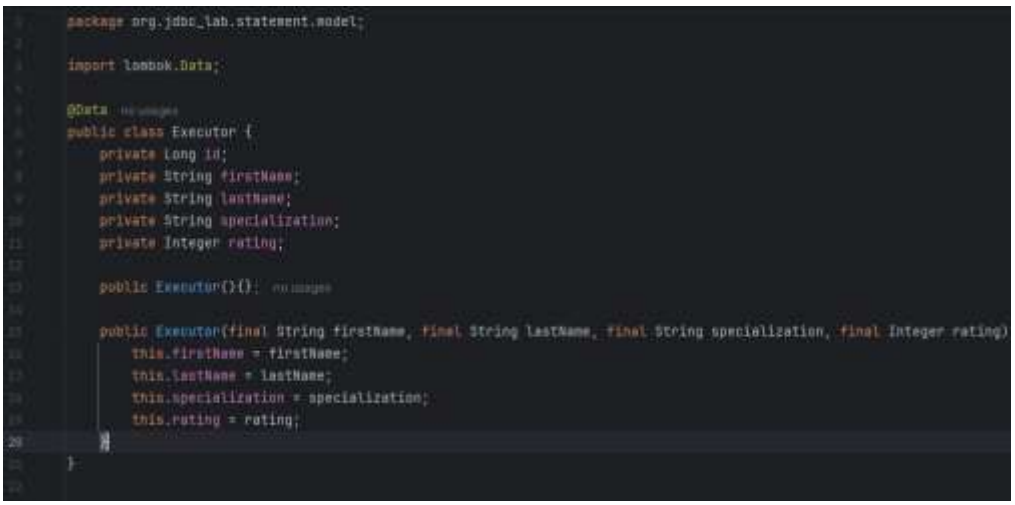
```
package org.jdbc_lab.statement.model;

import lombok.Data;

@Data
public class Client {
    private Long id;
    private String name;
    private String phone;
    private String email;

    public Client() {}

    public Client(String name, String phone, String email) {
        this.name = name;
        this.phone = phone;
        this.email = email;
    }
}
```



```java
package org.jdbc_lab.statement.model;

import lombok.Data;

@Data no usages
public class Executor {
    private Long id;
    private String firstName;
    private String lastName;
    private String specialization;
    private Integer rating;

    public Executor(){} no usages

    public Executor(final String firstName, final String lastName, final String specialization, final Integer rating)
        this.firstName = firstName;
        this.lastName = lastName;
        this.specialization = specialization;
        this.rating = rating;
    }
}
```

```java
package org.jdbc_lab.statement.model;

import lombok.Data;

@Data 15 usages
public class Order {
    private Long id;
    private Long clientId;
    private Long serviceId;
    private String orderDate;
    private String status;

    public Order() {} 2 usages

    public Order(Long clientId, Long serviceId, String orderDate, String status) { 3 usages
        this.clientId = clientId;
        this.serviceId = serviceId;
        this.orderDate = orderDate;
        this.status = status;
    }
}
```

```java
package org.jdbc_lab.statement.model;

import lombok.Data;

@Data no usages
public class Service {
    private Long id;
    private String title;
    private Long executorId;
    private Integer price;
    private String category;

    public Service() {} no usages

    public Service(String title, Long executorId, Integer price, String category) { no usages
        this.title = title;
        this.executorId = executorId;
        this.price = price;
        this.category = category;
    }
}
```

Для инкапсуляции логики работы с базой данных создадим интерфейсы StatementDao и PreparedStatementDao в пакетах dao. В интерфейсах объявим базовые методы CRUD-операций (insert, update, delete, getAll, getById), а затем реализуем их в конкретных классах доступа к данным. Внутри методов будем использовать объекты Statement или PreparedStatement для выполнения SQL-запросов и обрабатывать ResultSet для преобразования строк базы данных в Java-объекты.

```java
package org.jdbc_lab.statement.dao;

import java.util.List;

public interface StatementDao<T> {  4 usages  4 implementations
    void clearTable();  4 usages  4 implementations
    void insert(T t);  4 implementations
    int getCount();  4 usages  4 implementations
    List<T> getAll();  4 usages  4 implementations
    T getById(Long id);  no usages  4 implementations
}
```

```java
package org.jdbc_lab.prepared_statement.dao;

import java.util.List;

public interface PreparedStatementDao<T> {  1 usage  1 implementation
    Rename usages
    void clearTable();  1 implementation
    void insert(T t);  1 implementation
    int getCount();  1 usage  1 implementation
    List<T> getAll();  1 usage  1 implementation
    T getById(Long id);  no usages  1 implementation

}
```

Далее создадим классы доступа к данным для каждой сущности. В пакете statement реализуем классы ExecutorDao, ServiceDao, ClientDao и OrderDao, каждый из которых реализует интерфейс StatementDao для соответствующего типа. Внутри методов будем использовать объект Statement, получаемый через ConnectionManager.getConnection().createStatement(), и формировать SQL-запросы путём конкатенации строк со значениями полей объектов.

```
package org.jdbc_lab.statement.dao;

import org.jdbc_lab.connection.ConnectionManager;
import org.jdbc_lab.statement.model.Executor;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;
import java.util.logging.Logger;

public class ExecutorDao implements StatementDao<Executor> {
    private static final Logger logger = Logger.getLogger(ExecutorDao.class.getName());

    @Override
    public void clearTable(){
        try (Statement stmt = ConnectionManager.getConnection().createStatement()){
            stmt.executeUpdate("DELETE FROM executors");
        }
        catch (SQLException e){
            logger.severe("Ошибка: " + e.getMessage());
            throw new RuntimeException(e);
        }

    }

    @Override
    public void insert(Executor executor) {
        try (Statement stmt = ConnectionManager.getConnection().createStatement()) {
            stmt.executeUpdate("INSERT INTO executors (first_name, last_name, specialization, rating) " +
                    "VALUES ('" + executor.getFirstName() + "', '" + executor.getLastName() + "', '" +
                    executor.getSpecialization() + "', " + executor.getRating() + ")");
        } catch (SQLException e) {
            logger.severe("Ошибка: " + e.getMessage());
            throw new RuntimeException(e);
        }
    }

    @Override
    public int getCount(){
        int count = 0;

        try (Statement stmt = ConnectionManager.getConnection().createStatement()){
            ResultSet resultSet = stmt.executeQuery("SELECT COUNT(*) FROM executors");
            if (resultSet.next()) {
                count = resultSet.getInt(1);
            }
        } catch (SQLException e) {
            logger.severe("Ошибка: " + e.getMessage());
            throw new RuntimeException(e);
        }

        return count;
    }

    @Override
    public List<Executor> getAll() {
        List<Executor> executors = new ArrayList<>();

        try (Statement stmt = ConnectionManager.getConnection().createStatement()) {
            ResultSet resultSet = stmt.executeQuery("SELECT * FROM executors");
```

```
            while (resultSet.next()) {
                Executor executor = new Executor();
                executor.setId(resultSet.getLong("id"));
                executor.setFirstName(resultSet.getString("first_name"));
                executor.setLastName(resultSet.getString("last_name"));
                executor.setSpecialization(resultSet.getString("specialization"));
                executor.setRating(resultSet.getInt("rating"));
                executors.add(executor);
            }
        } catch (SQLException e) {
            logger.severe("Ошибка: " + e.getMessage());
            throw new RuntimeException(e);
        }
        return executors;
    }

    @Override
    public Executor getById(Long id) {
        Executor executor = null;

        try (Statement stmt = ConnectionManager.getConnection().createStatement()) {
            ResultSet resultSet = stmt.executeQuery("SELECT * FROM executors WHERE id = " + id);

            if (resultSet.next()) {
                executor = new Executor();
                executor.setId(resultSet.getLong("id"));
                executor.setFirstName(resultSet.getString("first_name"));
                executor.setLastName(resultSet.getString("last_name"));
                executor.setSpecialization(resultSet.getString("specialization"));
                executor.setRating(resultSet.getInt("rating"));
            }
        } catch (SQLException e) {
            logger.severe("Ошибка: " + e.getMessage());
            throw new RuntimeException(e);
        }

        return executor;
    }
}
```

В пакете prepared_statement создадим классы MechanicDao, CarDao и RepairJobDao, реализующие интерфейс PreparedStatementDao. Здесь будем использовать объект PreparedStatement с плейсхолдерами (?), подставляя значения параметров через методы setString, setInt и setLong, что обеспечивает защиту от SQL-инъекций.

```
package org.jdbc_lab.prepared_statement.dao;

import org.jdbc_lab.connection.ConnectionManager;
import org.jdbc_lab.prepared_statement.model.Car;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;
import java.util.logging.Logger;

public class CarDao {
    private static final Logger logger = Logger.getLogger(CarDao.class.getName());

    public void insert(Car car) {
```

```
        try (PreparedStatement stmt = ConnectionManager.getConnection()
                .prepareStatement("INSERT INTO cars (brand, model, year, license_plate) VALUES (?, ?, ?, ?)")) {
            stmt.setString(1, car.getBrand());
            stmt.setString(2, car.getModel());
            stmt.setInt(3, car.getYear());
            stmt.setString(4, car.getLicensePlate());
            stmt.executeUpdate();
        } catch (SQLException e) {
            logger.severe("Ошибка при добавлении авто: " + e.getMessage());
            throw new RuntimeException(e);
        }
    }

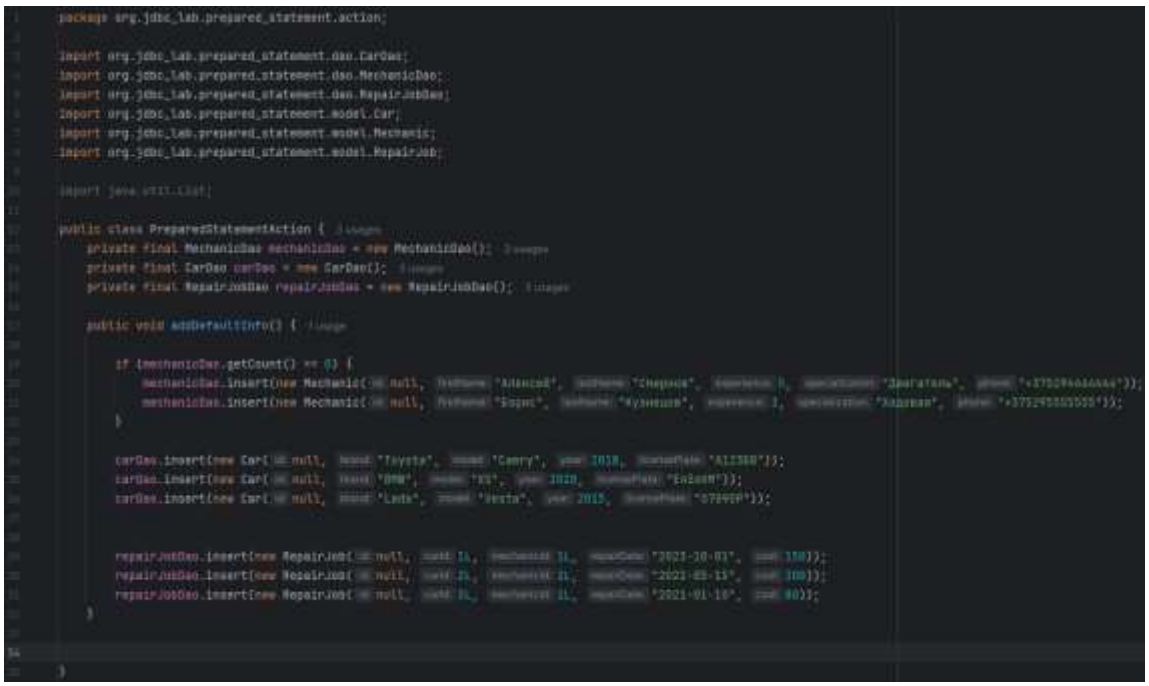
    public List<Car> getAll() {
        List<Car> cars = new ArrayList<>();
        try (PreparedStatement stmt = ConnectionManager.getConnection()
                .prepareStatement("SELECT * FROM cars")) {
            ResultSet rs = stmt.executeQuery();
            while (rs.next()) {
                Car car = new Car();
                car.setId(rs.getLong("id"));
                car.setBrand(rs.getString("brand"));
                car.setModel(rs.getString("model"));
                car.setYear(rs.getInt("year"));
                car.setLicensePlate(rs.getString("license_plate"));
                cars.add(car);
            }
        } catch (SQLException e) {
            logger.severe("Ошибка при получении авто: " + e.getMessage());
            throw new RuntimeException(e);
        }
        return cars;
    }
}
```

Для реализации бизнес-логики и заданий из методички создадим классы StatementAction и PreparedStatementAction в пакетах action. В классе StatementAction напишем методы для выполнения 7 заданий: выборка по всем таблицам, сортировка исполнителей, фильтрация услуг по цене, обновление статуса случайного заказа, поиск услуг по категории, обновление телефонов клиентов и группировка услуг. В классе PreparedStatementAction реализуем методы для работы с автосервисом: добавление тестовых данных, поиск ремонтов по механику, обновление стоимости и удаление старых записей.



```java
public class StatementAction {
    private final ExecutorDao executorDao = new ExecutorDao();
    private final ServiceDao serviceDao = new ServiceDao();
    private final ClientDao clientDao = new ClientDao();
    private final OrderDao orderDao = new OrderDao();

    public void deleteAllInfo() {
        executorDao.clearTable();
        serviceDao.clearTable();
        clientDao.clearTable();
        orderDao.clearTable();
    }
    public void addDefaultInfo() {
        if (executorDao.getCount() == 0) {
            executorDao.insert(new Executor( firstName: "Иван", lastName: "Петров", specialization: "Сантехник", rating: 5));
            executorDao.insert(new Executor( firstName: "Анна", lastName: "Сидорова", specialization: "Электрик", rating: 4));
            executorDao.insert(new Executor( firstName: "Сергей", lastName: "Иванов", specialization: "Плотник", rating: 5));
        }
        if (clientDao.getCount() == 0) {
            clientDao.insert(new Client( name: "Мария", phone: "+375291111111", email: "maria@mail.ru"));
            clientDao.insert(new Client( name: "Дмитрий", phone: "+3752922222222", email: "dmitry@mail.ru"));
            clientDao.insert(new Client( name: "Елена", phone: "+375293333333", email: "elena@mail.ru"));
        }
        if (serviceDao.getCount() == 0) {
            serviceDao.insert(new Service( title: "Ремонт труб", executorId: 1L, price: 100, category: "Сантехника"));
            serviceDao.insert(new Service( title: "Замена розетки", executorId: 2L, price: 50, category: "Электрика"));
            serviceDao.insert(new Service( title: "Сборка мебели", executorId: 3L, price: 150, category: "Плотницкие работы"));
        }
        if (orderDao.getCount() == 0) {
            orderDao.insert(new Order( clientId: 1L, serviceId: 1L, orderDate: "2026-09-15", status: "Завершён"));
            orderDao.insert(new Order( clientId: 2L, serviceId: 2L, orderDate: "2026-09-16", status: "В работе"));
            orderDao.insert(new Order( clientId: 3L, serviceId: 3L, orderDate: "2026-09-17", status: "Новый"));
        }
    }
    public void getAllInfo() {
        System.out.println("\n=== ИСПОЛНИТЕЛИ ===");
        List<Executor> executors = executorDao.getAll();
        for (Executor executor : executors) {
            System.out.println(executor);
        }
        System.out.println("\n=== КЛИЕНТЫ ===");
        List<Client> clients = clientDao.getAll();
        for (Client client : clients) {
            System.out.println(client);
        }
        System.out.println("\n=== УСЛУГИ ===");
        List<Service> services = serviceDao.getAll();
        for (Service service : services) {
            System.out.println(service);
        }
        System.out.println("\n=== ЗАКАЗЫ ===");
        List<Order> orders = orderDao.getAll();
        for (Order order : orders) {
            System.out.println(order);
        }
    }
}
```



Statement

Для выполнения первого задания сделаем выборку по всем таблицам базы данных. Для этого создадим метод selectAllTables() и добавим его в файл StatementAction.java.

```
public class StatementAction {
    public void getAllInfo() {
        System.out.println("\n=== ИСПОЛНИТЕЛИ ===");
        List<Executor> executors = executorDao.getAll();
        for (Executor executor : executors) {
            System.out.println(executor);
        }

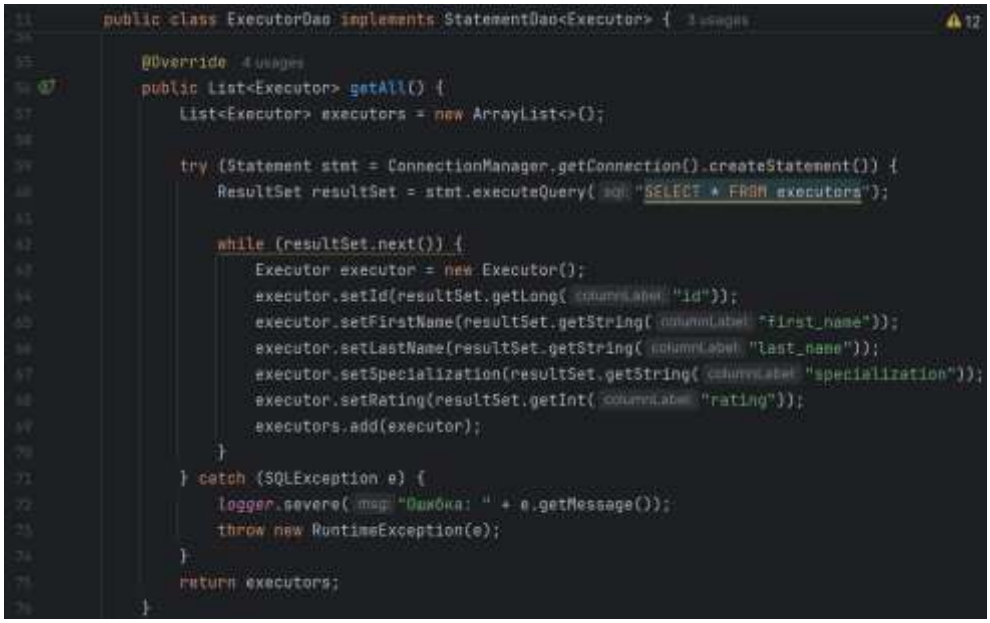
        System.out.println("\n=== КЛИЕНТЫ ===");
        List<Client> clients = clientDao.getAll();
        for (Client client : clients) {
            System.out.println(client);
        }

        System.out.println("\n=== УСЛУГИ ===");
        List<Service> services = serviceDao.getAll();
        for (Service service : services) {
            System.out.println(service);
        }

        System.out.println("\n=== ЗАКАЗЫ ===");
        List<Order> orders = orderDao.getAll();
        for (Order order : orders) {
            System.out.println(order);
        }
    }
```

Внутри метода вызовем методы getAll() у всех DAO-классов (ExecutorDao, ServiceDao, ClientDao, OrderDao) и выведем полученные

результаты в консоль. Также добавим соответствующий пункт меню для вызова этого метода в файл StatementMenu.java.



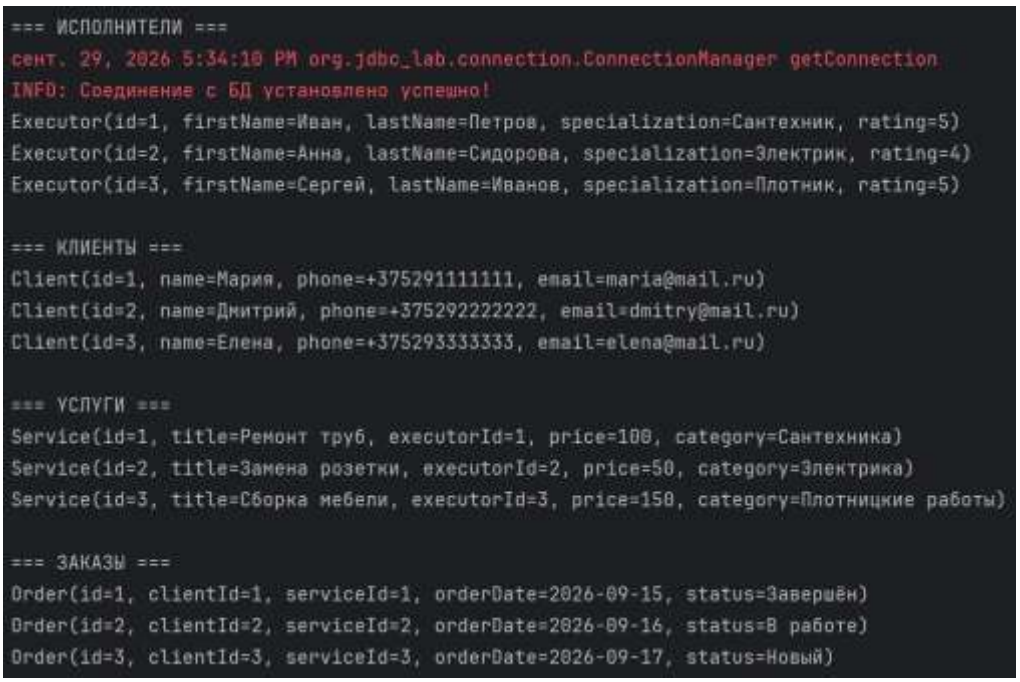
```
public class ExecutorDao implements StatementDao<Executor> {
    @Override
    public List<Executor> getAll() {
        List<Executor> executors = new ArrayList<>();

        try (Statement stmt = ConnectionManager.getConnection().createStatement()) {
            ResultSet resultSet = stmt.executeQuery("SELECT * FROM executors");

            while (resultSet.next()) {
                Executor executor = new Executor();
                executor.setId(resultSet.getLong("id"));
                executor.setFirstName(resultSet.getString("first_name"));
                executor.setLastName(resultSet.getString("last_name"));
                executor.setSpecialization(resultSet.getString("specialization"));
                executor.setRating(resultSet.getInt("rating"));
                executors.add(executor);
            }
        } catch (SQLException e) {
            logger.severe("Ошибка: " + e.getMessage());
            throw new RuntimeException(e);
        }
        return executors;
    }
```



```
public class StatementMenu {
    public void print() {
                    case 2:
                        statementAction.getAllInfo();
                        break;
```



```
=== ИСПОЛНИТЕЛИ ===
сент. 29, 2026 5:34:10 PM org.jdbc_lab.connection.ConnectionManager getConnection
INFO: Соединение с БД установлено успешно!
Executor(id=1, firstName=Иван, lastName=Петров, specialization=Сантехник, rating=5)
Executor(id=2, firstName=Анна, lastName=Сидорова, specialization=Электрик, rating=4)
Executor(id=3, firstName=Сергей, lastName=Иванов, specialization=Плотник, rating=5)

=== КЛИЕНТЫ ===
Client(id=1, name=Мария, phone=+375291111111, email=maria@mail.ru)
Client(id=2, name=Дмитрий, phone=+375292222222, email=dmitry@mail.ru)
Client(id=3, name=Елена, phone=+375293333333, email=elena@mail.ru)

=== УСЛУГИ ===
Service(id=1, title=Ремонт труб, executorId=1, price=100, category=Сантехника)
Service(id=2, title=Замена розетки, executorId=2, price=50, category=Электрика)
Service(id=3, title=Сборка мебели, executorId=3, price=150, category=Плотницкие работы)

=== ЗАКАЗЫ ===
Order(id=1, clientId=1, serviceId=1, orderDate=2026-09-15, status=Завершён)
Order(id=2, clientId=2, serviceId=2, orderDate=2026-09-16, status=В работе)
Order(id=3, clientId=3, serviceId=3, orderDate=2026-09-17, status=Новый)
```

Для выполнения второго задания сделаем выборку исполнителей, отсортировав их по имени и фамилии. Создадим метод getSortedExecutors() в файле StatementAction.java, который будет выполнять SQL-запрос с

конструкцией ORDER BY last_name, first_name. Добавим вызов этого метода в StatementMenu.java.

```
public class StatementAction {  3 usages
    public void getExecutorsSortedByName() {  1 usage
        System.out.println("\n=== ИСПОЛНИТЕЛИ (отсортированные по фамилии и имени) ===");
        List<Executor> executors = executorDao.getAllSortedBySurname();
        for (Executor executor : executors) {
            System.out.println(executor);
        }
    }
```

```
public class StatementMenu {  4 usages
    public void print() {  1 usage
                    case 4:
                        statementAction.getExecutorsSortedByName();
                        break;
```

```
=== ИСПОЛНИТЕЛИ (отсортированные по фамилии и имени) ===
сент. 29, 2026 5:49:26 PM org.jdbc_lab.connection.ConnectionManager getConnection
INFO: Соединение с БД установлено успешно!
Executor(id=3, firstName=Сергей, lastName=Иванов, specialization=Плотник, rating=5)
Executor(id=1, firstName=Иван, lastName=Петров, specialization=Сантехник, rating=5)
Executor(id=2, firstName=Анна, lastName=Сидорова, specialization=Электрик, rating=4)
```

Для выполнения третьего задания сделаем выборку услуг с ценой выше заданной. Создадим метод getServicesByPrice() в файле StatementAction.java, который будет принимать пороговое значение и выполнять запрос с условием WHERE price > ... ORDER BY price. Добавим пункт меню в StatementMenu.java.

```
public class StatementAction {  3 usages
    public void getServicesByPriceGreaterThan(Scanner scanner) {  1 usage
        System.out.print("Введите цену, выше которой нужно найти услуги: ");
        int price = scanner.nextInt();
        scanner.nextLine();

        System.out.println("\n=== УСЛУГИ (цена выше " + price + ", отсортированы по цене) ===");
        List<Service> services = serviceDao.getByPriceGreaterThan(price);
        for (Service service : services) {
            System.out.println(service);
        }
    }
```

```
Введите цену, выше которой нужно найти услуги: 100

=== УСЛУГИ (цена выше 100, отсортированы по цене) ===
Service(id=3, title=Сборка мебели, executorId=3, price=150, category=Плотницкие работы)
```

Для выполнения четвертого задания обновим статус случайного заказа конкретного клиента. Создадим метод updateRandomOrderStatus() в файле

StatementAction.java, который будет выполнять запрос UPDATE ... ORDER BY RANDOM() LIMIT 1. Добавим пункт меню в StatementMenu.java.

```
public class StatementAction {  3 usages
    public void updateRandomOrderStatus(Scanner scanner) {  1 usage
        System.out.print("Введите ID клиента: ");
        long clientId = scanner.nextLong();
        scanner.nextLine();

        System.out.print("Введите новый статус заказа: ");
        String newStatus = scanner.nextLine();

        orderDao.updateRandomOrderStatus(clientId, newStatus);
        System.out.println("Статус случайного заказа успешно обновлён!");
    }
```

```
Введите ID клиента: 1
Введите новый статус заказа: Новый
Статус случайного заказа успешно обновлён!
```

Для выполнения пятого задания сделаем выборку всех услуг, категория которых начинается на определённую букву. Создадим метод getServicesByCategoryStartingWith(Scanner scanner) в файле StatementAction.java, использующий оператор LIKE 'буква%'. Добавим пункт меню в StatementMenu.java.

```
public class StatementAction {  3 usages
    public void getServicesByCategoryStartingWith(Scanner scanner) {  1 usage
        System.out.print("Введите первую букву специализации (категории): ");
        String letter = scanner.nextLine();

        System.out.println("\n=== УСЛУГИ (категория начинается на '" + letter + "') ===");
        List<Service> services = serviceDao.getByCategoryStartingWith(letter);
        for (Service service : services) {
            System.out.println(service);
        }
    }
```

```
=== МЕНЮ STATEMENT (Маркетплейс услуг) ===
1. Добавить информацию по умолчанию
2. Просмотреть всю информацию
3. Очистить таблицы
4. Показать исполнителей (сортировка по фамилии и имени)
5. Найти услуги дороже заданной цены
6. Обновить статус случайного заказа клиента
7. Найти услуги по первой букве категории
8. Обновить телефон клиентов по статусу заказа
9. Показать количество услуг по категориям
0. Выйти в главное меню
Выберите пункт: 7
Введите первую букву специализации (категории): П

=== УСЛУГИ (категория начинается на 'П') ===
Service(id=3, title=Сборка мебели, executorId=3, price=150, category=Плотницкие работы)
```

Для выполнения шестого задания обновим телефоны клиентов, чьи заказы находятся в определённом статусе, с помощью подзапроса. Создадим метод updateClientPhoneByOrderStatus(Scanner scanner) в файле StatementAction.java, использующий конструкцию UPDATE ... WHERE id IN (SELECT client_id FROM orders WHERE status = ...). Добавим пункт меню в StatementMenu.java.

```
117 @       public void updateClientPhoneByOrderStatus(Scanner scanner) {  1 usage
118             System.out.print("Введите новый телефон: ");
119             String newPhone = scanner.nextLine();
120
121             System.out.print("Введите статус заказов (например, 'В работе'): ");
122             String status = scanner.nextLine();
123
124             clientDao.updateClientPhoneByOrderStatus(newPhone, status);
125             System.out.println("Телефоны клиентов успешно обновлены!");
126         }
```

```
=== МЕНЮ STATEMENT (Маркетплейс услуг) ===
1. Добавить информацию по умолчанию
2. Просмотреть всю информацию
3. Очистить таблицы
4. Показать исполнителей (сортировка по фамилии и имени)
5. Найти услуги дороже заданной цены
6. Обновить статус случайного заказа клиента
7. Найти услуги по первой букве категории
8. Обновить телефон клиентов по статусу заказа
9. Показать количество услуг по категориям
0. Выйти в главное меню
Выберите пункт: 8
Введите новый телефон: +375291653761
Введите статус заказов (например, 'В работе'): В работе
Телефоны клиентов успешно обновлены!
```

Для выполнения седьмого задания сделаем выборку услуг, сгруппировав её по категориям, и выведем количество услуг для каждой категории. Создадим метод getServicesCountByCategory() в файле StatementAction.java, использующий GROUP BY category и COUNT(*). Добавим пункт меню в StatementMenu.java.

```
128         public void getServicesCountByCategory() {  1 usage
129             System.out.println("\n=== КОЛИЧЕСТВО УСЛУГ ПО КАТЕГОРИЯМ ===");
130             List<String> result = serviceDao.getServicesCountByCategory();
131             for (String line : result) {
132                 System.out.println(line);
133             }
134         }
```

```
=== КОЛИЧЕСТВО УСЛУГ ПО КАТЕГОРИЯМ ===
Сантехника — 1 услуг(и)
Электрика — 1 услуг(и)
Плотницкие работы — 1 услуг(и)
```

PreparedStatement

Для выполнения первого задания добавим новые записи в таблицы. Создадим метод addDefaultInfo() в файле PreparedStatementAction.java, который будет использовать DAO-классы для вставки тестовых данных. Добавим соответствующий пункт меню в файл PreparedStatementMenu.java.



```
=== МЕНЮ PREPARED STATEMENT (Автосервис) ===
1. Добавить тестовые данные (Механики, Машины, Ремонты)
2. Показать ремонты конкретного механика
3. Обновить стоимость ремонта
4. Удалить старые ремонты
0. Выйти в главное меню
Выберите пункт: 1
сент. 29, 2026 6:36:04 PM org.jdbc_lab.connection.ConnectionManager getConnection
INFO: Соединение с БД установлено успешно!
Тестовые данные успешно добавлены!
```

Для выполнения второго задания сделаем поиск ремонтов по идентификатору механика. Создадим метод showRepairsByMechanic() в файле PreparedStatementAction.java, который будет принимать ID механика и выполнять параметризованный запрос с условием WHERE mechanic_id = ?. Добавим пункт меню в PreparedStatementMenu.java.

```
public void showRepairsByMechanic(Long mechanicId) {  1 usage
    System.out.println("\n=== РЕМОНТЫ МЕХАНИКА С ID = " + mechanicId + " ===");
    List<RepairJob> jobs = repairJobDao.getByMechanicId(mechanicId);

    if (jobs.isEmpty()) {
        System.out.println("У этого механика пока нет записей о ремонтах.");
    } else {
        for (RepairJob job : jobs) {
            System.out.println(job);
        }
    }
}
```

```
=== МЕНЮ PREPARED STATEMENT (Автосервис) ===
1. Добавить тестовые данные (Механики, Машины, Ремонты)
2. Показать ремонты конкретного механика
3. Обновить стоимость ремонта
4. Удалить старые ремонты
0. Выйти в главное меню
Выберите пункт: 2
Введите ID механика (например, 1 или 2): 1

=== РЕМОНТЫ МЕХАНИКА С ID = 1 ===
RepairJob(id=1, carId=1, mechanicId=1, repairDate=2023-10-01, cost=150)
RepairJob(id=3, carId=3, mechanicId=1, repairDate=2021-01-10, cost=80)
```

Для выполнения третьего задания обновим стоимость ремонта. Создадим метод updateRepairCost() в файле PreparedStatementAction.java, который будет принимать ID ремонта и новую цену, выполняя запрос UPDATE ... SET cost = ? WHERE id = ?. Добавим пункт меню в PreparedStatementMenu.java.

```
public void updateRepairCost(Long jobId, Integer newCost) {  1 usage
    repairJobDao.updateCost(jobId, newCost);
    System.out.println("Стоимость ремонта с ID=" + jobId + " успешно обновлена на " + newCost);
}
```

```
=== МЕНЮ PREPARED STATEMENT (Автосервис) ===
1. Добавить тестовые данные (Механики, Машины, Ремонты)
2. Показать ремонты конкретного механика
3. Обновить стоимость ремонта
4. Удалить старые ремонты
0. Выйти в главное меню
Выберите пункт: 3
Введите ID ремонта для обновления: 1
Введите новую стоимость: 170
Стоимость ремонта с ID=1 успешно обновлена на 170
```

Для выполнения четвертого задания удалим старые записи о ремонтах. Создадим метод deleteOldRepairs() в файле PreparedStatementAction.java, который будет принимать дату и выполнять запрос DELETE FROM repair_jobs WHERE repair_date < ?. Добавим пункт меню в PreparedStatementMenu.java.

```
public void deleteOldRepairs(String date) {  1 usage
    int count = repairJobDao.deleteOlderThan(date);
    System.out.println("Удалено старых ремонтов (до даты " + date + "): " + count + " шт.");
}
```

```
=== МЕНЮ PREPARED STATEMENT (Автосервис) ===
1. Добавить тестовые данные (Механики, Машины, Ремонты)
2. Показать ремонты конкретного механика
3. Обновить стоимость ремонта
4. Удалить старые ремонты
0. Выйти в главное меню
Выберите пункт: 4
Введите дату (например, 2022-01-01), чтобы удалить ремонты до неё: 2024-03-03
Удалено старых ремонтов (до даты 2024-03-03): 3 шт.
```

Вывод: В результате выполнения работы закреплены теоретические знания и получены практические навыки работы с базами данных на языке Java с использованием технологии JDBC. Были реализованы CRUD-операции, изучены различия между Statement и PreparedStatement, а также получены навыки написания сложных SQL-запросов